

The intuition — "FUSE talks straight to S3, so it should be more consistent" — is reasonable, and it inverts. Consistency is not proximity to the store; it is arbitration. The hub *is* the arbiter, and removing it is what direct mode trades away. This page asks both modes the same eight questions.

why “closer to S3” does not mean “more consistent”

- 1** consistency is manufactured, not inherited

S3 is strongly consistent per OBJECT. A filesystem is many objects plus metadata — ordering across them requires an authority. Direct mode deliberately runs without one.
- 2** the “extra hop” was the serialization point

NFS’s round-trip through the hub is exactly where opens, locks and renames get ordered. Removing the hop removes the ordering, not just the latency.
- 3** the write path never got closer to S3

Both modes ack on local media and publish at flush cadence — the same tier engine. Direct mode moved the working set into your pod; it did not move S3 closer.

the same eight questions, both modes

	hub — NFS over S3	direct — FUSE over S3
your own writes, your mount	strong POSIX — every op a network round-trip	strong POSIX — local disk speed; faster, not stronger
another client / pod reads your file	close-to-open — enforced by the hub, every client	RPO-stale snapshot — at most one flush interval + one revalidate old
two writers, one file	serialized — locks and share reservations are real	out of contract — one writer per subtree, lease-enforced
locks · O_EXCL · rename	real across every client of the share	ENOLCK · O_EXCL as opt-in sync PUT · EXDEV across subtrees
an acked fsync survives...	pod and node death — the PVC holds it	container crashes only — emptyDir dies with the pod
what an S3 browser sees	the last flush barrier — untorn whole-file objects	identical — same engine, same cadence
staleness when things break	reads HANG — D-state until the hub comes back	reads ERROR (ENOTCONN); your own subtree keeps serving
after a crash + restart	everything acked — dirty files re-publish whole	published only — plus a 60–110s lease lockout after unclean death

the stronger cell

the weaker / bounded cell

tie

score: every cross-client row belongs to the hub; direct wins latency and failure shape — the S3 rows tie.

where direct mode genuinely is better

write latency — no network round-trip, no sunrpc per-connection ceiling on the data path

failure shape — staleness and faults surface as errno, never as a D-state hang

blast radius — one pod's staleness is one pod's problem

identical in both — it is the same engine

ack ≠ in S3: both modes are write-back; S3 trails by the flush cadence (floor 60s) in hub mode and direct mode alike

untorn, CAS-guarded whole-file generations; S3 itself is strong per object (2020-12) — neither mode changes that

console PUTs against a live tree: the same three fates

the upgrades — and the ceiling

- strict journal mode (both modes): fsync ⇒ in S3 — group-commit PUT, ~10–50 ms, per-path policy
- revalidate-on-open (direct): foreign reads upgrade to close-to-open — NFS parity for single-writer workspaces
- the ceiling: multi-writer coherence needs an arbiter that answers in ms — building one is rebuilding the hub

The precise statement: per S3 *object*, both modes are equally strong — S3 has been read-after-write consistent since 2020-12 and every flint publish is CAS-guarded and untorn, so "direct to S3" ties the rows the intuition was actually about. Per *filesystem*, the hub is strictly stronger everywhere two clients meet, because it is the meeting point: close-to-open, byte-range locks, O_EXCL, atomic rename, and multi-writer serialization all exist *because* every operation takes the extra hop the intuition wanted to remove. Direct mode is stronger only where nobody meets — your own subtree's latency, your independence from everyone else's failures, and the shape of failure itself (an error you can handle instead of a hang you cannot). **In one sentence: direct mode is a faster filesystem with weaker company; the hub is a slower filesystem with one shared truth.** With the two upgrades — the strict group-commit journal (fsync ⇒ durable in S3, ~10–50 ms, per-path policy) and revalidation-on-open — a single-writer direct-mode workspace reaches NFS-equivalent close-to-open with *better* durability than today's hub default. The one row that can never transfer is multi-writer coherence: an arbiter that answers in milliseconds is a server by definition, and flint already ships it — the hub.